Antigravity Code Generation Prompt

Milo Platform — Approval Workflow Engine (Phase 5)

Context

Milo is an autonomous AI program coordinator built on LangGraph, FastAPI, and AWS Lambda. It manages multi-stakeholder programs for SMEs. The platform is being built using the Milo_Platform_Build_Specification.docx v1.3.

Currently, Milo uses a `handoff__human` tool as a one-way escalation mechanism — it fires and forgets. There is no structured path for the human to respond, approve, reject, or delegate back to Milo. This breaks the autonomous coordination loop.

This prompt requests the full Approval Workflow Engine — a structured, bidirectional decision loop between Milo and the human owner.

Objective

Build an **Approval Workflow Engine** that allows Milo to:

1. Queue a structured approval request (with context, options, and deadline)
2. Notify the human via email (and later Slack)
3. Receive a structured response (approve / reject / delegate / defer)
4. Resume autonomous action based on the decision

New MCP Tools Required

1. `approval__create`

Queue a new approval request.

Input:

```
{
  "title": "string — short description of what needs approval",
  "description": "string — full context, background, and recommendation",
  "options": ["approve", "reject", "delegate", "defer"],
  "requested_by": "string — Milo or agent name",
  "notify_email": "string — email to notify",
  "due_by": "ISO 8601 datetime — when decision is needed by",
  "work_item_id": "string (optional) — linked work item UUID",
  "metadata": "object (optional) — any extra context"
}
```

Output:

```
{
  "approval_id": "uuid",
  "status": "pending",
```

```
  "created_at": "ISO 8601"
}
```

---

2. `approval__read`

Read the status of one or all pending approvals.

Input:

```
{
  "approval_id": "string (optional) — if omitted, returns all pending"
}
```

Output:

```
{
  "approvals": [
    {
      "approval_id": "uuid",
      "title": "string",
      "status": "pending | approved | rejected | delegated | deferred | expired",
      "decision": "string (optional)",
      "decided_by": "string (optional)",
      "decided_at": "ISO 8601 (optional)",
      "due_by": "ISO 8601",
      "work_item_id": "string (optional)"
    }
  ]
}
```

---

3. `approval__respond`

Submit a decision on a pending approval. Used by the human via UI or email reply parser.

Input:

```
{
  "approval_id": "uuid",
  "decision": "approved | rejected | delegated | deferred",
  "notes": "string (optional) — reason or instructions",
  "decided_by": "string — name or email of decision maker"
}
```

Output:

```
{
  "approval_id": "uuid",
  "status": "approved | rejected | delegated | deferred",
```

```
  "decided_at": "ISO 8601"
}
```

---

4. `approval__cancel`

Cancel a pending approval request (e.g. if the situation resolved itself).

Input:

```
{
  "approval_id": "uuid",
  "reason": "string (optional)"
}
```

Output:

```
{
  "approval_id": "uuid",
  "status": "cancelled"
}
```

Notification Behavior

When `approval__create` is called:

1. **Immediately** send an email to `notify_email` with:
   - Subject: `[Milo] Action Required: {title}`
   - Body: Full description, options, due date, and a **one-click response link** per option
   - Footer: `Reply with APPROVE, REJECT, DELEGATE, or DEFER to respond directly`
2. If `due_by` passes with no response → auto-set status to `expired` and fire a `trigger__evaluate` to escalate

---

Email Reply Parser

Build a lightweight inbound email parser that:

- Watches for replies to Milo approval emails (matching subject thread or approval_id token in body)
- Extracts decision keyword: `APPROVE`, `REJECT`, `DELEGATE`, `DEFER`
- Calls `approval__respond` automatically
- Sends confirmation email back to the human: `[Milo] Decision Recorded: {title} → {decision}`

Database Schema

Table: `approvals`

```
CREATE TABLE approvals (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL,
```

```
  title TEXT NOT NULL,
  description TEXT,
  options JSONB NOT NULL DEFAULT '["approve","reject"]',
  requested_by TEXT NOT NULL,
  notify_email TEXT NOT NULL,
  due_by TIMESTAMPTZ,
  status TEXT NOT NULL DEFAULT 'pending',
  decision TEXT,
  decided_by TEXT,
  decided_at TIMESTAMPTZ,
  notes TEXT,
  work_item_id UUID REFERENCES work_items(id),
  metadata JSONB,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);
```

API Endpoints

Method	Path	Description
POST	`/approvals`	Create approval (maps to `approval__create`)
GET	`/approvals`	List all approvals for tenant
GET	`/approvals/{id}`	Read single approval
POST	`/approvals/{id}/respond`	Submit decision (maps to `approval__respond`)
DELETE	`/approvals/{id}`	Cancel approval (maps to `approval__cancel`)
POST	`/approvals/inbound-email`	Webhook for inbound email reply parser

All endpoints require tenant JWT auth. Follow existing FastAPI router patterns in the Milo codebase.

Upgrade: Replace `handoff__human`

Update the existing `handoff__human` tool to:

1. Call `approval__create` internally
2. Return the `approval_id` so Milo can poll or be notified
3. Maintain backward compatibility (existing calls still work)

UI Requirements (Phase 5 Approvals UX)

Build a minimal **Approvals Queue** page in the Milo web frontend:

- List view: all pending approvals sorted by `due_by` ASC
- Each row: title, requested_by, due_by, status badge, action buttons (Approve / Reject / Delegate / Defer)
- Click → modal with full description and notes field
- Submit → calls `POST /approvals/{id}/respond`

- Real-time status update (polling or websocket)
- Mobile-friendly (you will likely approve from your phone)

Tech Stack & Conventions

Follow all existing Milo platform conventions:

- **Backend:** FastAPI, async SQLAlchemy, PostgreSQL (RDS), Alembic migrations
- **Auth:** JWT with tenant isolation on every query
- **Infrastructure:** AWS Lambda via CDK, API Gateway
- **Email:** Nylas MCP for send; inbound webhook for reply parsing
- **Testing:** pytest-asyncio, 90%+ coverage on approval state machine
- **Secrets:** AWS Secrets Manager (no hardcoded credentials)
- **Logging:** Structured JSON logs, Langfuse tracing on all agent turns

---

Acceptance Criteria

1. `approval__create` queues a record in DB and sends notification email within 5 seconds
2. `approval__read` returns correct status for all approval states
3. `approval__respond` updates status and fires confirmation email
4. Email reply parser correctly extracts APPROVE/REJECT/DELEGATE/DEFER from plain-text reply
5. Expired approvals auto-update status when `due_by` passes
6. `handoff__human` internally calls `approval__create` and returns `approval_id`
7. Approvals Queue UI renders all pending items sorted by due date
8. One-click approve/reject works from UI modal
9. All endpoints enforce tenant isolation (no cross-tenant data leakage)
10. Alembic migration included for `approvals` table
11. pytest suite covers: create, read, respond, cancel, expiry, email parse
12. CDK construct deploys cleanly to `milo-poc` AWS account

Priority

P1 — Phase 5 blocker. This is the single biggest gap in Milo's autonomy loop. Without it, every escalation is a dead end.

Deliverables

1. `services/approvals/router.py` — FastAPI router with all 5 endpoints
2. `services/approvals/models.py` — SQLAlchemy model + Alembic migration
3. `services/approvals/email_parser.py` — Inbound reply parser
4. `services/approvals/notifier.py` — Email notification sender
5. `tools/approval__create.py` — MCP tool stub
6. `tools/approval__read.py` — MCP tool stub
7. `tools/approval__respond.py` — MCP tool stub
8. `tools/approval__cancel.py` — MCP tool stub
9. `infra/approvals_stack.py` — CDK construct
10. `tests/test_approvals.py` — Full pytest suite
11. `frontend/pages/approvals.tsx` — Approvals Queue UI page
